Manuel d'Architecture du Projet Cascadia

Réalisé par : BEN ALLAK Hassina & HAMMI Fouad

Introduction
Objectifs du projet
Choix d'architecture
Contenu du rendu
compilation
Problèmes rencontrés
Conclusion

Introduction

Dans ce document on explique l'architecture qu'on a choisi pour notre projet, on décrit ses composants principaux, et tout les autres choix de programmation adoptés pour la réalisation de se dernier.

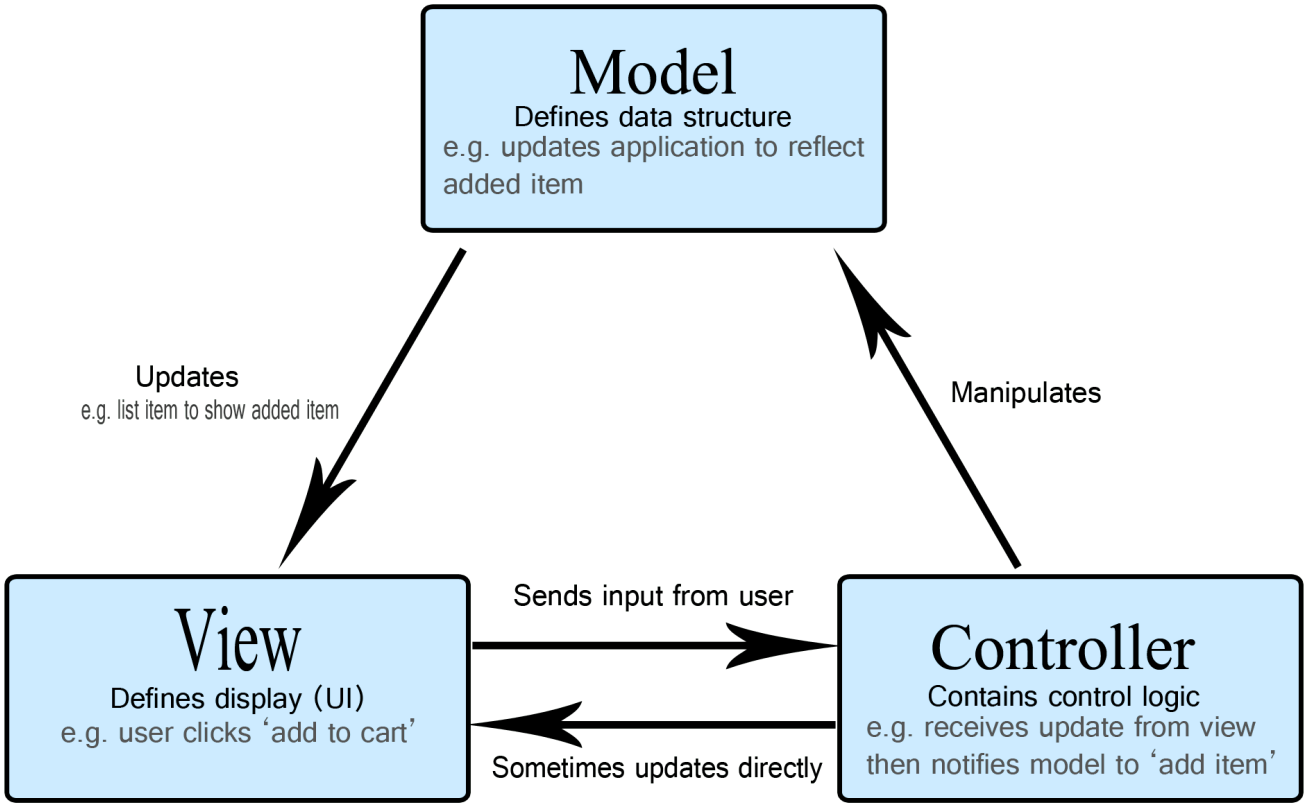
Objectifs du Projet

- **Créer une version en ligne du jeu de société Cascadia pour Bosphore :**
  On a développé une implémentation numérique du jeu de société Cascadia, proposant trois variantes différentes du jeu basées sur l'utilisation de tuiles carrées et hexagonales. Chaque version respectant plus ou moins les régles du jeu.
- **Offrir une expérience utilisateur agréable à bosphore :**
  L'objectif est que Bosphore puisse profiter pleinement du jeu. Cela inclut des instructions claires, et une représentation visuelle conforme aux règles du jeu.
- **Respecter les principes de programmation orientée objet :**
  Un autre objectif est d'utiliser une architecture modulaire et extensible qui permet la réutilisabilité et la lisibilité de notre code. Chaque composant de notre projet est encapsulé dans des classes ou des records distincts adaptées à leurs roles, et interagit via des interfaces lorsque c'est nécessaire, respectant ainsi les principes de modularité et d'encapsulation de la programmation objet.

- **Respecter les consignes :**
  L’objectif ultime est de respecter toutes les consignes spécifiées dans le sujet du projet, pour que notre projet remplit tous les critères d’évaluation.

Choix d'Architecture

- L'architecture utilisé suit le modèle **MVC (Modèle-Vue-Contrôleur)** pour séparer les responsabilités, améliorer et simplifier les tests.



Contenu du rendu

Le modèle contient toute la logique et les données du jeu. Voici ses composants principaux :

src

Les packages

- **`fr.uge.cascadia.structure` :** Ce package contient les classes fondamentales et nécessaires pour modéliser les éléments du jeu Cascadia. Il regroupe les structures principales qui permettent de représenter les éléments physiques du jeu ainsi que leur gestion.
  - **Classes principales :**
    a. **`Animal` :**
      - **Type** : `Enum`.
      - **Rôle** : Représente les animaux disponibles dans le jeu (ours, wapiti, saumon...)
      - **Usage** : Utilisé pour associer un animal à une tuile ou comme jeton faune.

    b. **`Habitat` :**
      - **Type** : `Enum`.
      - **Rôle** : Définit les différents types d'habitats du jeu (forêt, marais, prairie ...)
      - **Usage** : Associe un ou plusieurs habitats à une tuile et influence le placement des tuiles dans l'environnement.

    c. **`Tile` :**
      - **Type** : Classe.
      - **Rôle** : Modélise une tuile du jeu avec ses propriétés principales :
        - **Habitats** (tableau d'habitats).
        - **Animals** (tableau des animaux associés).
        - **Rotation** (int, pour gérer l'orientation des tuiles).
      - **Usage** : Représente un composant clé du plateau de jeu, utilisé dans les lots à choisir et lors du placement dans l'environnement.

    d. **`TileAnimalPair` :**
      - **Type** : Record.
      - **Rôle** : Associe une tuile Tile avec un jeton faune Animal.
      - **Usage** : Utilisé dans la gestion des lots disponibles pour les joueurs, et la représentation d'une tuile après qu'un joueur y place un jeton faune.

    e. **`Tiles` :**
      - **Type** : Classe.
      - **Rôle** : Représente l'ensemble des 85 tuiles habitat disponibles dans le jeu, gère la pioche et la défausse des tuiles, et initialise les tuiles habitat de départ.
      - **Usage** : Assure une gestion cohérente et aléatoire des tuiles durant une partie.

    f. **`FauneBag` :**
      - **Type** : Classe.

        - **Rôle** : Modélise le sac contenant les jetons faune, gère la pioche et la défausse des jetons, et fournit des mécanismes pour réinitialiser ou mélanger le sac.
        - **Usage** : Garantit une gestion équilibrée des jetons faune, selon les règles du jeu.
    g. `GameLot` **:**
        - **Type** : Classe.
        - **Rôle** : Gère les lots disponibles pour chaque tour, chaque lot contient une tuile habitat et un jeton faune associés.
        - **Usage** : Implémente les règles de remplacement ou de modification des lots.
    h. `Lot` **:**
        - **Type** : Interface.
        - **Rôle** : Définie les comportements communs pour les éléments qui peuvent être placés sur le plateau de jeu:
            - Tuiles habitat (Tile).
            - Tuiles associées à un jeton faune (TileAnimalPair).
        - **Usage** : Simplifie la gestion des éléments placés dans l'environnement du jeu.
- `fr.uge.cascadia.game` **:** Ce package contient les classes principales liées à la gestion de la logique du jeu Cascadia. Il inclut les mécanismes de gestion des joueurs, du plateau de jeu (environnement), des interactions, et le choix des options du jeu.
    - **Classes principales :**
    a. `Player` **:**
        - **Type** : Classe.
        - **Rôle** : Représente un joueur dans une partie, et stocke ses informations :
            - Nom
            - Environnement personnel (plateau).
            - Nombre de Jetons Nature
            - Calculateur de score
        - **Usage** : Assure le suivi individuel de chaque joueur, et permet d'interagir avec l'environnement (jouer un tour, calculer les score...)
    b. `Environnement` **:**
        - **Type** : Classe.
        - **Rôle** : Représente le plateau personnel de chaque joueur, et gère les placements des tuiles et jetons faune selon les règles du jeu.
          Fournit des méthodes comme pour Valider les placements selon les habitats et les animaux...
        - **Usage** : Permet une manipulation fluide des éléments du plateau (placement, rotation),et s'adapte au mode de jeu (tuiles hexagonales ou carrées).
    c. `GameManager` **:**
        - **Type** : Classe.

        - **Rôle** :
            - Initialise les éléments nécessaires :
                - Les tuiles et jetons disponibles.
                - Les joueurs.
            - Assure et contrôle le déroulement global d'une partie.
            - Assure la gestion des joueurs, des tours, et des interactions avec les lots disponibles.
            - Implémente les règles spécifiques selon les modes .
        - **Usage** :
            - Coordonne toutes les actions pendant la partie.
            - Gère les appels aux méthodes des autres classes, comme Player ou GameLot...
    d. `GameChoices` **:**
        - **Type** : Classe.
        - **Rôle** :
            - Gère les choix du joueur pour configurer la partie, comme :
                - Mode de jeu : dans le terminal, affichage graphique avec tuiles carrées ou avec tuiles hexagonales.
                - Variantes (famille ou intérmediaire)
                - Nombre de joueurs.
                - Cartes de score choisies.
            - Fournit des interactions utilisateur basées sur des entrées clavier (scanner) avec controle de saisie.
        - **Usage** :
            - Simplifie la configuration des choix du jeu avant son démarrage.
            - Assure une interface claire pour les options de jeu.

- `fr.uge.cascadia.view` **:** Ce package implémente la gestion de l'interface graphique et de l'affichage des éléments du jeu Cascadia. Il permet de visualiser les actions des joueurs dans leurs plateaux de jeu et les lots disponibles, en offrant une représentation graphique claire et fonctionnelle.
    - **Classes principales :**
        a. `GameViewSquare` **:**
        - **Type** : Classe.
        - **Rôle** :
            Gère l'affichage du plateau et des lots lorsque le jeu utilise des tuiles carrées, et Permet de dessiner :
            - Les tuiles carrées sur le plateau de chaque joueur.

        - Les lots disponibles avec leurs tuiles et jetons faune.
          Implémente des mécanismes pour les interactions des joueurs (choix des lots, placement des tuiles et des animaux).
    - **Usage** : Fournit une interface graphique adaptée pour le mode tuiles carrés.
  b. `GameViewHex` **:**
    - **Type** : Classe.
    - **Rôle** :
      Gère l'affichage du plateau et des lots lorsque le jeu utilise des tuiles hexagonales.
      Implémente des méthodes pour :
      - Dessiner les tuiles hexagonales avec gestion de la rotation.
      - Afficher les lots disponibles et gérer les clics des joueurs pour choisir ou placer des éléments.
      - Ajouter des boutons pour les actions spécifiques comme la rotation ou l'utilisation des jetons nature.
    - **Usage** : Fournit une interface graphique adaptée pour le mode tuiles hexagonale.
  c. `ImageLoader` **:**
    - **Type** : Classe.
    - **Rôle** :
      - Gère le chargement des images associées aux habitats et jetons faune.
      - Utilise un mécanisme de cache pour optimiser les performances et éviter de recharger plusieurs fois les mêmes images.
    - **Usage** : Simplifie l'accès aux ressources (images) nécessaires pour l'interface.
- `fr.uge.cascadia.score` **:** Ce package contient les classes et interfaces nécessaires pour gérer le système de calcul des scores dans le jeu
  - **Contenu principal :**
    - **ScoreRule (Interface)** : Définie une règle de score générique.
    - **MajorityBonus** : Classe qui calcule les bonus de majorité pour chaque habitat pour les joueur (soit mode Solo ou multijoueur).
    - **ScoreCalculator** : Classe responsable de la gestion des différents calculs de score pour les joueurs.
    - **BearScore (et les autres animaux)** : Classe spécifique pour chaque animal, comme l'ours (Bear), gérant ses règles de score selon la carte choisie au début de la partie (A,B,C,D).
    - **SuccessManager** et **PlayerProgress** : Classes pour gérer les succés Cascadia des joueurs.
- `fr.uge.cascadia.score.cards` **:** Ce sous-package contient les implémentations des différentes cartes de score pour chaque animal. Les cartes définissent les règles spécifiques de calcul des scores.

- **Contenu principal :**
  a. Structure générale :
    - Chaque animal possède plusieurs cartes de score **(A, B, C, D)** avec des règles spécifiques. Chaque classe implémente l'interface **ScoringRule**.

  b. Exemples de classes :
    - BearScoreCardA.
    - BuzzardScoreCardA.
    - FoxScoreCardB.
- **Avantages de cette structure :**
  - **Modularité :** Ajout facile de nouvelles règles ou cartes de score.
  - **Lisibilité :** Chaque carte est isolée, ce qui rend son comportement facile à comprendre et aussi à modifier.

- `fr.uge.cascadia` **:**
  - **Classe principale :**
    - **Main.java :** La classe Main représente le point d'entrée du projet Cascadia. On l'a implémenter pour initialiser les ressources nécessaires et démarrer la partie du jeu Cascadia.
  - **Responsabilité de la classe Main :**
    La classe Main suit a une responsabilité unique : **Démarrer l'application.**
    Toutes les autres tâches sont gérés dans leurs classes spécialisées.

ressources

1. **Images :**
   - Le répertoire contient toutes les images nécessaires pour le rendu graphique du jeu, notamment :
     - **Images des habitats carrés :** Illustrations représentant les différents types d'habitats (forêt, marais, rivière, etc.) pour les tuiles carrées.
     - **Images des habitats hexagonaux :** Illustrations adaptées aux tuiles hexagonales avec des combinaisons de plusieurs habitats.
     - **Images des animaux :** Illustrations des animaux associés aux jetons faune (ours, wapiti, saumon, etc.).
2. **Success :**
   - Le répertoire dédié aux succès contient :
     - Les fichiers permettant de suivre les performances des joueurs et leurs accomplissements au cours des parties

lib

Le répertoire lib contient la bibliothèque externe utilisée pour le projet : **Zen Library**, ce répertoire joue un rôle central dans la gestion de l'interface.

docs

- Ce répertoire contiendra un sous répertoire **doc** qui sera créer lors de la compilation, et contiendra la JavaDoc.
- **dev.pdf :** Ce document qui explique les choix de developpement et l'architecture du projet.
- **user.pdf :** Manuel d'utilisation pour les joueur .

build.xml

Permet de :

- Compiler des sources.
- Créer le jar exécutable `Cascadia.jar` .
- Générer la `javadoc` , écrite en anglais.
- nettoyer le projet pour qu'il ne reste plus que les éléments demandés.
  Pour plus les détails voir compilation.

Cascadia.jar :

Jar exécutable pour son éxecution voir compilation.

Compilation

- Dans le répertoire rendu se trouve un fichier **build.xml** qui permet de créer l'éxecutable `Cascadia.jar` et **le répertoire bin** pour les fichiers **.class** en éxucutant dans un terminal la commande :
  `ant` ou `ant jar`
- On peut ensuite éxecuter `Cascadia.jar` avec la commande :
  `java -jar Cascadia.jar`
- Pour créer la **javadoc** dans le répertoire **/docs/doc** on doit éxecuter la commande :
  `ant javadoc`
- Pour consulter la **javadoc** il faut accéder au fichier **/docs/doc/index.html**.
- Pour nettoyer tout ça il suffit d'éxecuter la commande :
  `ant clean`

Difficultés

- Malgré nos efforts pour répondre a toutes les consignes et implémenter toutes les fonctionnalités demandées, nous avons rencontré quelques obstacles techniques qui nous ont empêchés de compléter certaines parties du projet.
- **Parties non réussies :**
  1. Gestion du remplacement des lots dans le mode solo :
  Lors du remplacement des lots dans le mode solo, nous devions :
  - Retirer le lot le plus éloigné de la pioche (le dernier).
  - Décaler les lots restants vers la droite.
  - Compléter avec de nouveaux lots.
  - **Difficulté technique :**
  L'implémentation de cette logique s'est avérée compliqué, notamment pour :
  - Gérer efficacement le décalage des lots.
  - Mettre à jour les indices des lots et des jetons de manière cohérente sans conflits.
  - Assurer que chaque joueur sélectionne uniquement les lots disponibles après le remplacement.
  - **Leçons apprises :**
  - La gestion dynamique des lots et des indices nécessite une attention particulière pour éviter les conflits et les incohérences.
  - Les manipulations sur des collections comme les listes doivent être bien planifiées, en particulier quand des éléments sont ajoutés, supprimés ou modifiés.
  2. les succés spéciaux Cascadia :
  Nous n’avons pas implémenté les succès spéciaux Cascadia, qui ajoutent des objectifs spécifiques supplémentaires pour les joueurs.
  - **Niveau de difficulté :**
  Cette fonctionnalité s'est avérée un peu plus complexe que prévu à cause des éléments suivants :
  - Définition des règles : Chaque succès spécial requiert des conditions spécifiques (ex. : alignements d'animaux, groupes d'habitats) qu'il faut définir.
  - Calcul des succès dynamiques : Intégrer des algorithmes pour détecter ces configurations sur le plateau des joueurs.
  L’absence de cette fonctionnalité ne bloque pas le fonctionnement du jeu, mais elle réduit la profondeur stratégique et la rejouabilité en solo et en multijoueur.

Conclusion

Le développement de Cascadia en ligne était un défi enrichissant, avec des concepts avancés de programmation orientée objet, de gestion graphique et d'interaction utilisateur.

Nous avons structuré le projet autour de principes de la modularité et la réutilisabilité du code, pour garantir une base solide. L'architecture en packages clairement définis permet une gestion efficace des responsabilités, simplifiant ainsi le développement, les tests et les améliorations futures.

Malgré certaines limitations, nous avons réussi à implémenter une grande partie des fonctionnalités demandées. Les modes de jeu avec tuiles carrées et hexagonales, l'intégration des règles spéciales avec les jetons Nature, ainsi que les différentes variantes de score .

Enfin, ce projet a permis de mettre en pratique des compétences variées, de la gestion des collections et des flux de données au rendu graphique et à l'implémentation de règles complexes.

Nous espérons que ce jeu, dans sa forme actuelle, apportera une expérience agréable et stimulante aux utilisateurs tout en posant les bases d’un projet évolutif.